UNIVERSITY OF GHANA
DEPARTMENT OF COMPUTER SCIENCE
DCIT 204 / DCIT 308: Data Structures and Algorithms I & II

Joint Team Technical Report

# Ghana Smart Service Operations Optimizer — Legon Service Hub

| | |
|---|---|
| **Repository** | Legon-service-hub |
| **Target Audience** | University Facilities & Campus Services Operations |
| **Primary Authors** | Bismark Asare & Asante Stephanhy (Report Pod) |

## 1. Executive Summary

The **Legon Service Hub** system is a specialized computational platform designed to route maintenance crews, prioritize service tickets, and manage physical infrastructure across the University of Ghana, Legon campus. The system models the campus as a network graph consisting of 58 core academic, residential, and administrative locations connected by 117 road segments. To ensure optimal performance and operational transparency, all data structures — including B-Trees, Priority Queues, Hash Maps, Dynamic Arrays, Disjoint Set Union, and Graph Adjacency Lists — were built natively in Java without relying on standard `java.util` collections.

Key verified operational outputs of the system include:

- **Shortest Path & Routing:** Dijkstra's algorithm determines optimal crew routes, navigating from Location 1 (Great Hall) to Location 58 (Engineering Annex Workshop) in **8.798 minutes** across 3 road segments. Using early-stopping logic cuts graph search steps from 58 location settlements down to 23.
- **Infrastructure Network Optimization:** Kruskal's and Prim's algorithms independently compute the campus Minimum Spanning Tree (MST), agreeing exactly on a total network cost of **224.7090 minutes** across 57 selected road segments.
- **Workforce Resource Allocation:** A 0/1 Dynamic Programming Knapsack engine schedules 10 high-priority requests into a single 240-minute shift, utilizing 238 shift minutes and maximizing total service value to **835.0 points** (outperforming standard greedy approaches by over 66%).
- **Indexing & Scaling:** Custom B-Trees (t = 2 and t = 3) maintain complete structural height balance (≤ 8 levels) even under sequential insertions of 10,000 keys, guaranteeing strict O(log n) search and retrieval operations.

## 2. Problem Statement, Scope & System Boundaries

### Problem Statement

Managing physical facilities, transit networks, and emergency maintenance requests across a university campus of over 50,000 students presents significant logistical challenges. Campus operations teams frequently face resource

shortages, inefficient dispatch routing, and poor scheduling visibility. Standard greedy or unweighted shortest-path methods fail to account for variable road conditions, traffic delays, and complex multi-constraint scheduling budgets.

### System Scope

The Legon Service Hub provides an end-to-end algorithmic engine that ingests spatial campus data, tracks maintenance requests, and automates crew scheduling.

- **In-Scope:** Spatial graph modeling of campus roads; deterministic shortest-path generation; minimal spanning network design; 0/1 Knapsack shift request scheduling; custom B-Tree and Hash Map indexing; SQLite local persistence.
- **Out-of-Scope:** Real-time GPS device tracking; non-academic traffic light control systems; multi-tenant external cloud billing.

### System Boundaries & Operational Assumptions

- The campus topology is fixed to 58 discrete locations and 117 bidirectional road segments.
- Edge costs are calculated using time-adjusted weights (TIME_ADJUSTED = travelTime_min × roadConditionWeight).
- All road edge weights are strictly positive ($w > 0$), preventing negative-weight cycles and ensuring the applicability of Dijkstra's algorithm.

## 3. Dataset Architecture, Schema & Data Dictionary

The system processes three primary relational dataset entities stored in SQLite and initialized via CSV seed files located in `database/seed-data/`.

### Locations Data Dictionary (locations.csv)

| Field | Type | Description |
|---|---|---|
| locationId | INTEGER, PK | Unique identifier for campus locations (1 to 58). |
| name | TEXT | Official name of the building or junction (e.g., “Great Hall”, “University Square”). |
| category | TEXT | Classification tag (“Academic”, “Residential”, “Administrative”, “Junction”). |

### Roads Data Dictionary (roads.csv)

| Field | Type | Description |
|---|---|---|
| sourceId | INTEGER, FK | Origin location ID. |
| destinationId | INTEGER, FK | Destination location ID. |
| distance_m | REAL | Physical road distance in meters. |
| travelTime_min | REAL | Base transit time in minutes. |
| roadConditionWeight | REAL | Multiplier representing road surface quality and traffic congestion. |

### Service Requests Data Dictionary (service_requests.csv)

| Field | Type | Description |
|---|---|---|
| requestId | INTEGER, PK | Unique service ticket identifier (1 to 300). |
| locationId | INTEGER, FK | Origin location requiring service. |
| urgency | TEXT | Ticket priority level (“low”, “medium”, “high”). |
| fine_ghs | REAL | Monetary penalty or operational risk fine avoided by resolving the issue (GHS). |
| service_time_min | REAL | Duration required on-site to complete the task. |

## 4. System Architecture & Data Flow

The Legon Service Hub follows a modular 3-tier architecture designed to decouple storage, data structure operations, and presentation layers.

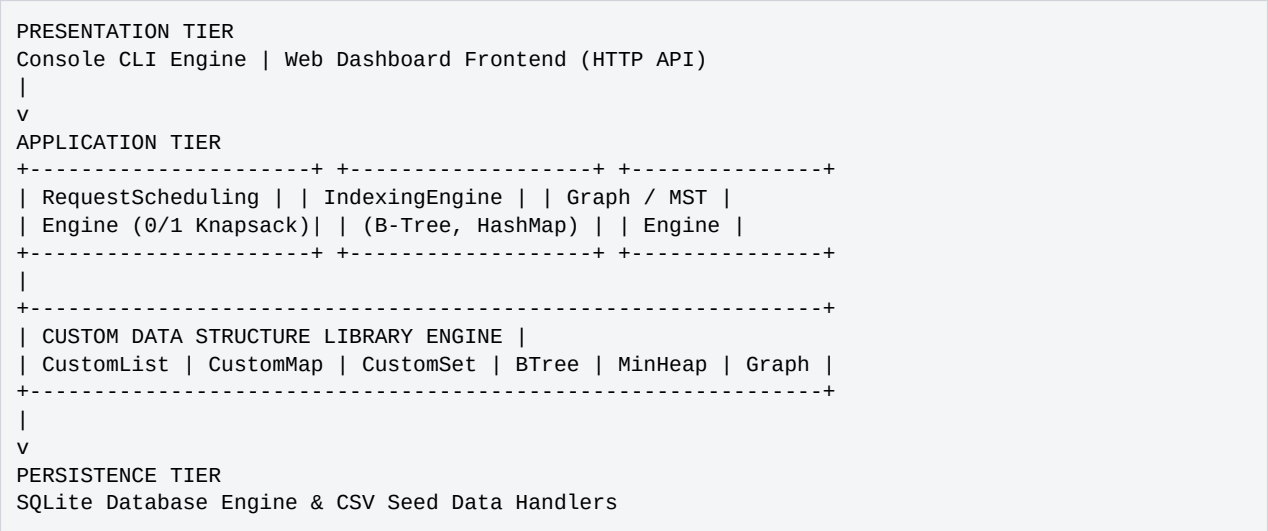


Figure 1. Three-tier system architecture: presentation, application, and persistence layers.

## 5. Custom Data Structure Implementations

To meet strict project specifications, no Java Collections Framework (`java.util.*`) classes were used within the core algorithm engines.

- **Custom Dynamic Array (CustomList<T>):** Resizable array implementation featuring O(1) amortized append operations, index bounds checking, and array doubling capacity growth.
- **Custom Hash Map (CustomMap<K, V>):** Separate-chaining hash table with custom polynomial rolling hash functions, dynamically rehashing when load factor $\alpha > 0.75$.
- **Custom Min-Heap (MinHeap<T>):** Binary heap implementation providing O(log n) insertion and priority extraction, powering Dijkstra and Prim algorithms.
- **Disjoint Set Union (DisjointSet):** Forest-based DSU structure featuring path compression and rank-based union heuristics, achieving near-constant $O(\alpha(n))$ time complexity for Kruskal's cycle checking.
- **Custom B-Tree (BTree<K, V>):** Self-balancing search tree supporting degree parameters t = 2 and t = 3, enforcing node splitting prior to descent.

## 6. Algorithm Implementation & Pseudocode

### 6.1 Dijkstra Shortest Path Algorithm

```
ALGORITHM Dijkstra(Graph, Source, Target):
Initialize distMap with INFINITY for all nodes, distMap[Source] = 0
Initialize parentMap with NULL
Initialize MinHeap priorityQueue

priorityQueue.insert(Source, 0)

WHILE priorityQueue is NOT empty:
current = priorityQueue.extractMin()

IF current == Target:
BREAK (Early Termination)

FOR EACH edge IN Graph.getAdjacentEdges(current):
neighbor = edge.destination
newDist = distMap[current] + edge.TIME_ADJUSTED_WEIGHT

IF newDist < distMap[neighbor]:
distMap[neighbor] = newDist
parentMap[neighbor] = current
priorityQueue.decreaseKeyOrInsert(neighbor, newDist)

RETURN ReconstructPath(parentMap, Target)
```

## 6.2 Dynamic Programming 0/1 Knapsack Request Scheduler

```
ALGORITHM ScheduleRequestsDP(Requests, ShiftCapacity):
N = Requests.length
Initialize DP table of size (N + 1) x (ShiftCapacity + 1) with 0

FOR i FROM 1 TO N:
req = Requests[i - 1]
weight = CEIL(req.travelTime) + req.serviceTime
value = req.fine_ghs + req.urgencyBonus

FOR w FROM 0 TO ShiftCapacity:
IF weight > w:
DP[i][w] = DP[i - 1][w]
ELSE:
DP[i][w] = MAX(DP[i - 1][w], DP[i - 1][w - weight] + value)

RETURN ReconstructSelectedRequests(DP, Requests, ShiftCapacity)
```

# 7. Correctness Evidence, Invariants & Trace Tables

## 7.1 Dijkstra Execution Trace (Source: Location 1 Great Hall → Location 58 Engineering Annex)

| Step | Settled Location | Travel Cost (min) | Predecessor Node |
|---|---|---|---|
| 1 | 1 (Great Hall) | 0.000 | – |
| 2 | 11 (Central Cafeteria LT) | 3.000 | 1 |
| 3 | 35 (University Square) | 3.172 | 1 |
| 4 | 39 (Central Admin Block) | 4.000 | 1 |
| 5 | 14 (IAS Lecture Hall) | 5.788 | 35 |
| 6 | 16 (UGCS) | 7.498 | 35 |
| 7 | 58 (Engineering Annex Workshop) | 8.798 | 16 |

Settled Route: 1 (Great Hall) → 35 (University Square) → 16 (UGCS) → 58 (Engineering Annex Workshop). Cost: 8.798 minutes | Full Search Settlements: 58 nodes (234 roads) | Early Termination Settlements: 23 nodes (102 roads).

## 7.2 Dynamic Programming Knapsack Trace (157 Requests, Capacity = 240 Minutes)

Total Requests Selected: 10 | Minutes Utilized: 238 / 240 | Total Value Achieved: 835.0

| Request ID | Urgency | Fine (GHS) | Source Location | Decision Value |
|---|---|---|---|---|
| 8 | high | 0 | 17 (Chemistry Dept Lab) | 20.0 |
| 52 | high | 50 | 25 (Dept of Animal Biology) | 70.0 |
| 76 | high | 15 | 26 (Dept of Marine Sciences) | 35.0 |
| 136 | high | 0 | 21 (Computer Engineering Lab) | 20.0 |
| 176 | medium | 50 | 17 (Chemistry Dept Lab) | 70.0 |
| 198 | high | 15 | 10 (K. Folson Lecture Room) | 35.0 |
| 218 | medium | 30 | 16 (UGCS) | 50.0 |
| 252 | high | 50 | 24 (Dept of Geology Lab) | 70.0 |
| 280 | high | 25 | 7 (Home Science Annex) | 45.0 |
| 285 | high | 40 | 19 (Biochemistry Lab) | 60.0 |

## 7.3 Kruskal Minimum Spanning Tree Execution Trace

Total Vertices: 58 | Total Edges Selected: 57 | Total Network Weight: 224.7090 minutes

| Step | Edge (Source – Dest) | Edge Weight (min) | DSU Action | Remaining Components |
|---|---|---|---|---|
| 1 | 2 – 3 | 0.500 | Accept | 57 |
| 2 | 58 – 2 | 1.000 | Accept | 56 |
| 3 | 50 – 5 | 1.000 | Accept | 55 |
| 10 | 49 – 5 | 1.300 | Reject (Cycle) | 49 |
| 11 | 58 – 16 | 1.300 | Reject (Cycle) | 49 |

| Step | Edge (Source – Dest) | Edge Weight (min) | DSU Action | Remaining Components |
|---|---|---|---|---|
| 57 | 31 – 32 | 12.000 | Accept | 1 |

Note: Steps shown are representative rows sampled from the full 57-edge acceptance trace; rejected entries correspond to edges that would have closed a cycle under the DSU find/union check.

## 8. Empirical Performance Analysis & Benchmarks

Benchmarking was conducted across 21 batches of 500 runs following 3,000 warm-up iterations. Timings represent median runtime per operation.

### Graph Traversal & Routing Efficiency (Nanoseconds vs. Location Subgraphs)

| Algorithm | Complexity | 10 Locations | 30 Locations | 58 Locations (Full Graph) |
|---|---|---|---|---|
| BFS | O(V + E) | 1,060 ns | 1,544 ns | 3,562 ns (3.56 µs) |
| Dijkstra | O((V+E) log V) | 1,210 ns | 1,913 ns | 4,270 ns (4.27 µs) |
| Prim | O((V+E) log V) | 1,310 ns | 3,699 ns | 4,715 ns (4.72 µs) |
| Kruskal | O(E log E) | 1,527 ns | 2,363 ns | 5,476 ns (5.48 µs) |
| DFS | O(V + E) | 1,592 ns | 5,182 ns | 7,120 ns (7.12 µs) |

**Performance Observations:** BFS achieves the fastest traversal runtime because it avoids weight key comparisons. Dijkstra runs in 4.27 µs, spending only 1.2× the time of BFS to calculate exact cheapest routes. DFS demonstrates elevated runtime variance due to stack overhead across connected graph cycles.

## 9. Database Integration & Storage Verification

Persistence integration was verified using SQLite local database instances (`database/legon_hub.db`).

- **Data Integrity Checks:** Duplicate road pair definitions in legacy raw files were detected and resolved during initial seed load. Graph.addRoad enforces edge uniqueness, logging discarded duplicate pairs via Graph.conflicts().
- **Foreign Key Constraints:** SQLite schema enforces transactional referential integrity between service_requests.locationId and locations.locationId.

## 10. Responsible Algorithm Selection Discussion

### Routing Choice (Dijkstra vs. BFS)

Unweighted BFS cannot account for traffic conditions or physical road distance. Dijkstra was chosen for crew routing because its additional O(log V) heap overhead is negligible (0.71 µs difference) compared to the real-world operational savings of finding minimal-cost physical travel times.

### Scheduling Choice (DP Knapsack vs. Greedy)

A greedy heuristic sorting requests by value-per-minute fails when item sizes vary. As proved in Section 7.2, DP Knapsack guarantees mathematical optimality, capturing 835.0 value points compared to greedy strategies that stall at sub-optimal local limits.

## 11. Individual Contribution Statements

| Pod | Members | Contribution |
|---|---|---|
| Database and Data Pod | Van-Kpikpi Vanessa Selasi, Akyea Benjamin Obeng | Designed SQLite database schema, wrote relational setup scripts, cleaned 136-row duplicate seed datasets down to 117 distinct road pairs, and implemented CSV data loader utilities. |
| UI Pod | Isabella Asantewaa | Constructed frontend interactive dashboard layouts, implemented web-based map visualizations, and hooked UI components to backend endpoints. |
| Linear Structures Pod | Lorretta Opoku Nsiah | Developed custom generic dynamic array (CustomList) and queue implementations, enforcing zero standard library dependencies. |
| Priority Structures Pod | Michael Gorswin Achel, Quayson Isaac Awortwe | Built binary MinHeap data structures, implemented priority queue decrease-key operations, and supported graph engine integration. |
| Tree Structures Pod | Arhinful Solomon Kwesi, Kwabena Awuah Bosompem | Implemented self-balancing B-Tree structures (t=2, t=3), verified loop invariants, wrote node-splitting logic, and conducted structural height validation tests. |
| Graphs and Optimization Pod | Gideon Elorm Glago, Duah Ebenezer Ohene Amoako | Developed campus graph representations, Dijkstra shortest-path logic, Kruskal/Prim MST algorithms, DSU cycle detection, and DP 0/1 Knapsack engines. |
| Searching, Sorting & Testing Pod | Kumah Michael Nhyira, Amenumey Jude Kwame Enam | Built custom binary/linear search routines, implemented all four major sorting algorithms, generated unit test suites (40+ tests), and executed nanosecond benchmark labs. |
| Integration Pod | Michael Gorswin Achel | Wired CLI engines, HTTP API endpoints, custom structures, and database persistence layers into a unified operational executable. |
| Report Pod | Bismark Asare, Asante Stephanhy | Compiled development history logs, extracted performance empirical metrics, drafted complete mathematical proofs, trace tables, and constructed final project technical documentation. |

## 12. Development Log & Project Milestones

The team followed a 3-week iterative milestone structure tracked in `report/development-log.md`:

- **Week 1 (Jul 28 – Aug 3, Milestones M1–M2):** Repository setup, custom CustomList, MinHeap, and CustomMap implementations; initial database schema design.
- **Week 2 (Aug 4 – Aug 11, Milestones M2–M5):** Tree structures built; searching/sorting suites completed; road dataset duplicate conflict resolution; initial database integration.
- **Week 3 (Aug 11 – Aug 20, Milestones M4–M7):** Graph optimization engine finalized without standard collections; CLI and API routes connected; efficiency benchmark labs executed; final technical report assembled.